1. DOCKER ANSWERS INDEPTH

What is Dockerfile?

Suppose we have an application that we want to run inside a Docker container.

A Dockerfile is a text file that contains step-by-step instructions for Docker to build a Docker image for that application.

In a Dockerfile, we define things such as which base image we need, which application files need to be copied, which dependencies need to be installed, which port the application uses, and what command should run when the container starts.

So, instead of manually preparing the application environment every time, we write all the required instructions inside the Dockerfile and Docker uses those instructions to create the image.

When we consider our application, we need some environment where the application can run. For example, a Node.js application needs Node.js, application files, dependencies, and a command to start the application.

EXAMPLE:

Let’s explain Dockerfile completely from the basics, without assuming much Docker knowledge. Let’s take just one example.

Suppose we created a simple Node.js application.

Our application has files such as:

package.json

app.js

Now, to run this application inside Docker, we need Node.js inside the image.

So, we create a file called:

Dockerfile

Inside that Dockerfile, we can write:

```
FROM node:20
```

This tells Docker to use the Node.js 20 image as the base image.

Next:

```
WORKDIR /app
```

This creates/sets /app as the working directory inside the image.

Next:

```
COPY package*.json ./
```

This copies the package files from our application into the /app directory inside the image.

Then:

```
RUN npm install
```

This installs the Node.js dependencies required by our application.

Next:

```
COPY . .
```

This copies the remaining application files into the /app directory inside the image.

Finally:

```
CMD ["node", "app.js"]
```

This tells Docker that when a container is started from this image, it should run node app.js.

So our complete Dockerfile looks like:

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "app.js"]
```

Now we have a Dockerfile, but the Dockerfile itself is not the image and it is not the container.

We need to use the Dockerfile to build an image.

We run:

```
docker build -t myapp .
```

This command tells Docker to read the Dockerfile from the current directory and build an image called myapp.

After the build is completed:

Dockerfile → docker build → Docker Image

Now the myapp image is ready.

Next, we can create and start a container from that image using:

```
docker run -d --name mycontainer myapp
```

Now the application runs inside the container.

Simple flow:

Dockerfile → docker build → Docker Image → docker run → Docker Container → Application Running

If we want to see the image:

```
docker images
```

If we want to see the running container:

```
docker ps
```

If we want to stop the container:

```
docker stop mycontainer
```

Main point:
Dockerfile tells Docker how to build an image for an application.
Docker Image is created using the instructions written inside the Dockerfile.
Docker Container is created and started from that image.
👉 Dockerfile = Instructions to build an image
👉 Docker Image = Packaged application template
👉 Docker Container = Running instance of the image
First understand the point “Why do we need a Dockerfile?” Once that is clear, we can understand each Dockerfile instruction such as FROM, WORKDIR, COPY, RUN, CMD, EXPOSE, and ENTRYPOINT one by one.
;;

What is Multi-stage Docker?

Multi-stage Docker does not mean running two applications inside one container, and it also does not mean running two images.
Simple ga:
Multi-stage build means using multiple FROM stages in one Dockerfile, where we build the application in the first stage and then copy only the required files into the final stage to create a small production image.
Example:
Stage 1 → Build application
Stage 2 → Final lightweight image
So the main purpose is reducing image size + keeping unnecessary build tools out of the final image.
Example:

```
FROM node:20 AS build
# application build

FROM nginx:alpine
# build files copy
```

👉 One Dockerfile → multiple build stages → one final Docker image.
An image can contain unnecessary files, but a container does not contain unnecessary containers. A container is a running instance of an image.
In a multi-stage build:
Build stage image → Node.js, npm, source code, build tools, etc. are present.
Final image → Only the files required for production are copied.
So Node.js, npm, source code, build tools, and similar things will not be present in the final image if they are not required at runtime.
Example:
Stage 1: node:20 → app build → lots of files/tools
↓ only dist copy
Stage 2: nginx:alpine → dist files only
↓
Final image → small & clean
👉 So the main purpose of a multi-stage build is to prevent unnecessary files/tools from entering the final image and reduce the image size.
Important: In multi-stage builds, intermediate stages/images are used for the build process; finally, what we run is one container from the final image.

Multi-stage Docker does not mean running two applications inside one container, and it also does not mean running two images.
Simple ga:
Multi-stage build means using multiple FROM stages in one Dockerfile, where we build the application in the first stage and then copy only the required files into the final stage to create a small production image.
Example:
Stage 1 → Build application
Stage 2 → Final lightweight image
So the main purpose is reducing image size + keeping unnecessary build tools out of the final image.
Example:
FROM node:20 AS build
# application build

FROM nginx:alpine
# build files copy
👉 One Dockerfile → multiple build stages → one final Docker image.

Example: Let’s understand it easily with a React/Node app example.
Suppose our application needs Node.js to build the application, but Node.js is not required in production; the built files are served by Nginx.
Dockerfile:

```
# Stage 1 - Build
FROM node:20 AS build

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build


# Stage 2 - Production
FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html
```

Flow:
Stage 1: Node image → dependencies install → application build → dist files create
Stage 2: Nginx image → dist files created in Stage 1 are copied → final image ready
Then:

```
docker build -t myapp .
docker run -d -p 8080:80 myapp
```

👉 Finally, one image (myapp) is created, and we run one container. In multi-stage builds, we use multiple images as build stages, but finally we get a small image without unnecessary files.

1. ____

1. What is Docker Image?

A Docker Image is a packaged, read-only template that contains everything required to run an application, such as application files, dependencies, libraries, configuration, and the required runtime environment.
A Docker Image itself is not a running application. It is used as a template to create Docker containers.
When we consider our application, first we need to package everything required for that application into an image. After the image is created, we can use that image to create one or more containers.
EXAMPLE:
Let's understand Docker Image completely from the basics with one example.
Suppose we created a Node.js application.
Our application requires Node.js, application files, npm dependencies, and a command to start the application.
Instead of installing all these things manually every time, we can package them into a Docker Image.
For example, we create an image called:
myapp
We can build the image using:

```
docker build -t myapp .
```

This command reads the Dockerfile and creates a Docker Image called myapp.
Now the image is ready.
But the image itself is not running.
To actually run the application, we create a container from the image:

```
docker run -d --name mycontainer myapp
```

Now the myapp image is used as a template to create the mycontainer container.
We can also create another container from the same image:

```
docker run -d --name mycontainer2 myapp
```

Now:
myapp → Image
mycontainer → Container
mycontainer2 → Container
Both containers are created from the same image.
So one Docker Image can be used to create multiple containers.
Simple flow:
Dockerfile → docker build → Docker Image → docker run → Docker Container
If we want to see the available images:

```
docker images
```

If we want to remove an image:

```
docker rmi myapp
```

But an image cannot normally be removed while it is still being used by existing containers.
Main point:
Docker Image = Packaged template used to create containers.
Dockerfile = Instructions used to build the image.
Container = Running instance created from the image.

👉 Dockerfile → Creates Image

👉 Image → Creates Container

👉 Container → Runs Application
;;

2. What is Docker Container?

A Docker Container is a running or stopped instance of a Docker Image.
The Docker Image contains the application and everything required to run it, but the application actually runs inside the container.
When we consider our application, the image is like a template, and the container is the actual instance created from that template.
EXAMPLE:
Suppose we already created an image called:
myapp
Now we create a container from that image:

```
docker create --name mycontainer myapp
```

This command creates the container but does not start it.
To start the container:

```
docker start mycontainer
```

Now the application starts running inside the container.
Instead of using two commands, we can also create and start the container together using:

```
docker run -d --name mycontainer myapp
```

The important difference is:
docker create → Creates the container only.
docker start → Starts an existing stopped container.
docker run → Creates and starts a new container.
We can see running containers using:

```
docker ps
```

We can see all containers, including stopped containers, using:

```
docker ps -a
```

If we want to stop the container:

```
docker stop mycontainer
```

If we want to start it again:

```
docker start mycontainer
```

If we want to remove it:

```
docker rm mycontainer
```

Simple flow:
Docker Image → Create Container → Start Container → Application Running
Main point:
Docker Image = Template.
Docker Container = Instance created from the image.
👉 docker create = Create container
👉 docker start = Start existing container
👉 docker run = Create + Start container
;;

3. What is Docker Volume?

Docker Volume is used to store data outside the container's writable layer so that the data can persist even when the container is removed.
Suppose we have a database running inside a Docker container.
The database stores important information such as users, products, orders, and other application data.
If the database data exists only inside the container and the container is removed, we can lose that data.
This is where Docker Volume is used.
EXAMPLE:
Suppose we have a MySQL database container.
First, we create a volume:

```
docker volume create mydata
```

Now we run MySQL and attach the volume:

```
docker run -d --name mysql -v mydata:/var/lib/mysql mysql
```

Here:
mydata → Docker Volume.
/var/lib/mysql → Directory inside the container where MySQL stores its data.
Now MySQL stores its persistent data in the volume.
Suppose we remove the MySQL container:

```
docker rm -f mysql
```

The container is removed, but the mydata volume can still exist.
We can create another MySQL container and attach the same volume.
So the data can continue to exist independently of the original container.
We can see volumes using:

```
docker volume ls
```

We can inspect a volume using:

```
docker volume inspect mydata
```

Simple flow:
Container → Docker Volume → Persistent Data
Main point:
Docker Volume = Persistent storage for container data.

👉 Container can be removed

👉 Volume can remain

👉 Data can be preserved

;;

4. What is Docker Network?

Docker Network is used to provide communication between containers and between containers and external networks.

Suppose our application has three parts:

Frontend

Backend

Database

Each part is running inside a separate container.

The frontend needs to communicate with the backend, and the backend needs to communicate with the database.

For this communication, Docker Networking is used.

EXAMPLE:

Suppose we create a network:

```
docker network create mynetwork
```

Now we create the backend container:

```
docker run -d --name backend --network mynetwork mybackend
```

Then we create the database container:

```
docker run -d --name database --network mynetwork mysql
```

Now both containers are connected to the same Docker network.

The backend can communicate with the database using the container/service name:

```
database:3306
```

Instead of depending on a fixed container IP address, applications can use Docker's built-in name resolution on user-defined networks.

We can see Docker networks using:

```
docker network ls
```

We can inspect the network using:

```
docker network inspect mynetwork
```

Simple flow:

Frontend Container → Docker Network → Backend Container → Docker Network → Database Container

Main point:

Docker Network = Allows containers to communicate with each other.

;;

5. What is Docker Compose?

Docker Compose is a Docker tool used to define multiple containers/services and their configurations in a single compose.yaml file and manage those containers together.

Suppose we have a website application. That website has one frontend, one backend, and one database.

Docker Compose allows us to define all these services in one file and create, configure, start, stop, and manage them together.

EXAMPLE:

Let's explain Docker Compose completely from the basics, without assuming much Docker knowledge. Let's take just one example.

Suppose we created a shopping website.

That website has 3 parts:

Frontend → The user can see the website here.

Backend → It processes the requests coming from the user.

Database → It stores user/data details.

Now, if we use Docker, we can run these 3 parts in 3 separate containers.

That means:

Frontend → Container 1

Backend → Container 2

Database → Container 3

Now what is the problem?

Every time, we need to start the 3 containers separately.

```
docker run frontend
docker run backend
docker run database
```

Along with that, we also need to manage configurations such as ports, network, environment variables, and other settings.

This is where Docker Compose is used.

With Docker Compose, we create one file and tell Docker:

"My application needs a frontend, backend, and database. Configure them like this."

The file name is usually:

compose.yaml

Once the file is ready, we simply run:

```
docker compose up -d
```

This is one command.

Then Docker Compose creates and starts the required containers/services.

So we do not need to give 3 separate commands every time.

Simple flow:

compose.yaml → docker compose up → 3 containers start → complete application running

Now, if we want to stop the entire application:

```
docker compose down
```

So, we do not have to stop/remove the 3 containers separately.

Main point:

Dockerfile = Image preparation.

Docker Compose = Managing multiple containers together.

;;

6. What is Docker Registry?

A Docker Registry is a storage system used to store and distribute Docker Images.

Suppose we build an application image on our local computer.

That image is available locally, but our production server may also need the same image.

Instead of building the image again on the production server, we can push the image to a Docker Registry.

EXAMPLE:

Suppose we create:

myapp

We build it:

```
docker build -t myapp .
```

Then we tag the image:

```
docker tag myapp username/myapp:latest
```

Now we push the image:

```
docker push username/myapp:latest
```

The image is stored in the registry.

Now the production server can download it:

```
docker pull username/myapp:latest
```

After pulling the image, we can run:

```
docker run -d username/myapp:latest
```

Simple flow:

Local Machine → Build Image → Push → Registry → Pull → Production Server

A registry can be public or private.

Main point:

Docker Registry = Central place for storing and distributing Docker Images.

;;

7. What is Docker Hub?

Docker Hub is a cloud-based Docker Image Registry provided by Docker.

Suppose we create an application image and want to share it with another developer or deploy it to a production server.

We can push the image to Docker Hub.

EXAMPLE:

Suppose we build:

myapp

```
docker build -t myapp .
```

Then we tag it with our Docker Hub repository:

```
docker tag myapp username/myapp:latest
```

We log in:

```
docker login
```

Then push:

```
docker push username/myapp:latest
```

Now the image is available in the Docker Hub repository.

Another machine can download it:

```
docker pull username/myapp:latest
```

Then run it:

```
docker run -d username/myapp:latest
```

So Docker Hub works as a central cloud registry for storing and sharing Docker Images.

Main point:

Docker Hub = Cloud-based Registry for storing and sharing Docker Images.

;;

8. What is Docker Build Context?

Docker Build Context is the set of files and directories that Docker can access during an image build.

Suppose our project contains:

Dockerfile
package.json
app.js
src/
node_modules/

When we run:

```
docker build -t myapp .
```

The . means the current directory is being used as the build context.

Docker can access files inside that context for instructions such as COPY.

For example:

```
COPY app.js /app/app.js
```

works because app.js is inside the build context.

Docker cannot simply use arbitrary files outside the build context through COPY.

A large build context can also make the build process less efficient.

This is why .dockerignore is commonly used to exclude unnecessary files.

Simple flow:

Project Directory → Build Context → Docker Build → Image

Main point:

Docker Build Context = Files and directories available to Docker during the image build.

;;

9. What is Docker CMD vs ENTRYPOINT?

CMD and ENTRYPOINT are Dockerfile instructions used to define what should run when a container starts, but they behave differently.

CMD defines the default command or default arguments for a container.

ENTRYPOINT defines the main executable of a container.

EXAMPLE:

Suppose we have a Node.js application and we want the application to start using:

```
node app.js
```

We can write:

```
CMD ["node", "app.js"]
```

When we run:

```
docker run myapp
```

Docker starts the container and runs the default command.

The important point is that CMD can be overridden by providing another command during docker run.

For example:

```
docker run myapp node test.js
```

In this case, the command supplied at runtime replaces the image's default CMD.

Now consider ENTRYPOINT.

Suppose we have a Python application and we want Python to always be the main executable.

We can write:

```
ENTRYPOINT ["python"]
```

Then:

```
CMD ["app.py"]
```

Now the resulting command is:

```
python app.py
```

If we run:

```
docker run myapp test.py
```

the result becomes:

```
python test.py
```

Here ENTRYPOINT provides the main executable and CMD provides the default argument.

Simple flow:

ENTRYPOINT → Main executable

CMD → Default command/arguments

Main point:

CMD = Default command or default arguments.

ENTRYPOINT = Main executable.

;;

10. What is Docker COPY vs ADD?

COPY and ADD are Dockerfile instructions used to bring files into a Docker Image, but ADD has some additional behavior.

COPY is mainly used to copy files or directories from the build context into the image.

ADD can also copy files and provides additional features such as extracting supported local tar archives.

EXAMPLE:

Suppose our application has:

index.html

We can write:

```
COPY index.html /usr/share/nginx/html/index.html
```

This means Docker takes index.html from the build context and copies it into the image.

Another example:

```
COPY . /app
```

This copies files from the build context into /app, subject to .dockerignore.

Now suppose we have:

app.tar.gz

We can write:

```
ADD app.tar.gz /app
```

For a supported local tar archive, Docker can extract the archive into the destination.

ADD can also copy normal files.

However, when we simply want to copy files from the build context, COPY is generally preferred because it has simpler and more explicit behavior.

Main point:

COPY = Straightforward file copying.

ADD = Copying plus additional features such as local tar archive extraction.

;;

11. What is Docker .dockerignore?

.dockerignore is a file used to exclude unnecessary files and directories from the Docker build context.

Suppose our project contains:

node_modules

.git

logs

temp

large development files

We usually do not need these files in the Docker build context.

We can create:

```
.dockerignore
```

and write:

```
node_modules
.git
logs
temp
```

Now Docker excludes these files from the build context.

This is especially useful when we use:

```
COPY . /app
```

because unnecessary files are not included in the build context and therefore are not copied by that instruction.

It can also help prevent accidentally including development-only or sensitive files in the build context.

Simple flow:

Project Files → .dockerignore → Unnecessary Files Excluded → Docker Build

Main point:

.dockerignore = Excludes unnecessary files from the Docker build context.

;;

12. What are Docker Environment Variables?

Docker Environment Variables are configuration values provided to a container so that the application can use different settings without changing the image.

Suppose our backend application needs to know the database host.

Instead of hard-coding the database host inside the application, we can use:

DB_HOST

We can provide it while starting the container:

```
docker run -d -e DB_HOST=database mybackend
```

Now the application inside the container can read:

```
DB_HOST=database
```

The same image can be used in different environments.

For example:

Development → DB_HOST=dev-database

Testing → DB_HOST=test-database

Production → DB_HOST=prod-database

We do not need to create a different image just because the configuration value changes.

Environment variables can also be defined through Docker Compose.

Main point:

Environment Variables = Runtime configuration values provided to the application.

;;

13. What is Docker Port Mapping?

Docker Port Mapping is used to connect a port on the Docker host to a port inside a container.

Suppose our application is running inside the container on port 80.

We want users to access it through port 8080 on the host.

We can run:

```
docker run -d -p 8080:80 myapp
```

Here:

8080 → Host port

80 → Container port

The flow is:

User → Host Port 8080 → Container Port 80 → Application

Now when the user accesses:

localhost:8080

the request is forwarded to port 80 inside the container.

The general format is:

```
-p HOST_PORT:CONTAINER_PORT
```

Main point:

Docker Port Mapping = Connect a host port to a container port.

;;

14. What is Docker Bind Mount?

A Docker Bind Mount allows a specific file or directory from the Docker host to be mounted directly into a container.

Suppose we are developing an application on our laptop.

Our source code is located at:

/home/user/myapp

We want the container to access this same directory.

We can run:

```
docker run -v /home/user/myapp:/app myapp
```

Here:

/home/user/myapp → Directory on the host

/app → Directory inside the container

Now the container can access the host files through /app.

If we modify files on the host, those changes can be visible inside the container because the same host directory is mounted.

Bind mounts are commonly useful during development.

Simple flow:

Host Directory → Bind Mount → Container Directory

Main point:

Bind Mount = Mount a specific host file or directory directly into a container.

;;

15. What is Docker Named Volume?

A Docker Named Volume is a Docker-managed storage location with a specific name.

Suppose we have a database container and we want its data to survive container recreation.

We can create:

```
docker volume create mydata
```

Then attach it:

```
docker run -d --name database -v mydata:/var/lib/mysql mysql
```

Here:

mydata → Named Volume

/var/lib/mysql → Database data directory inside the container

Docker manages the actual storage location of the volume.

If we remove the container:

```
docker rm -f database
```

the named volume can still remain.

We can then attach the same volume to another suitable container.

This makes named volumes useful for persistent application data.

Main point:

Named Volume = Docker-managed persistent storage identified by a name.

;;

16. What is Docker Bridge Network?

A Docker Bridge Network allows containers on the same Docker network to communicate with each other.

Suppose we have:

Backend Container

Database Container

We create:

```
docker network create mynetwork
```

Then:

```
docker run -d --name backend --network mynetwork mybackend
docker run -d --name database --network mynetwork mysql
```

Now both containers are connected to the same user-defined bridge network.

The backend can communicate with the database using:

database:3306

Docker provides name-based communication on user-defined networks.

This is useful when multiple containers need to communicate on the same Docker host.

Simple flow:

Backend → Bridge Network → Database

Main point:

Bridge Network = Network that allows containers on the same Docker network to communicate.

;;

17. What is Docker Swarm?

Docker Swarm is Docker's native orchestration technology used to manage services across a cluster of Docker hosts.

Suppose our production application is too important to depend on only one server.

We have:

Server 1

Server 2

Server 3

We can join these Docker hosts into a Swarm cluster.

A Swarm has manager nodes that maintain cluster state and schedule workloads, and worker nodes that run assigned workloads.

Suppose we want three replicas of our web application.

We can create:

```
docker service create --name web --replicas 3 nginx
```

Swarm manages the three replicas across the cluster.

If a task fails, Swarm can create a replacement to maintain the desired number of replicas.

We can check services:

```
docker service ls
```

We can check service tasks:

```
docker service ps web
```

We can scale:

```
docker service scale web=5
```

Now Swarm tries to maintain five replicas.

Simple flow:

Multiple Docker Hosts → Swarm Cluster → Services → Replicas

Main point:

Docker Swarm = Docker's native orchestration system for managing services across multiple Docker hosts.

;;

18. What is Docker Service?

A Docker Service is a definition of the desired state of an application workload in Docker Swarm.

Suppose we have a web application and we want three copies of it running.

Instead of manually creating three containers, we can create:

```
docker service create --name web --replicas 3 nginx
```

Here:

web → Service name
3 → Desired number of replicas
nginx → Image

Swarm creates and manages the tasks needed to maintain those replicas.

If one task fails, Swarm can create another task to return to the desired state.

We can check:

```
docker service ls
```

We can see tasks:

```
docker service ps web
```

We can scale:

```
docker service scale web=5
```

Simple flow:

Service Definition → Swarm → Tasks/Containers → Application Running

Main point:

Docker Service = Defines the desired state of an application workload in Swarm.

;;

19. What is Docker Stack?

Docker Stack is used to deploy a multi-service application to a Docker Swarm cluster using a Compose-format YAML file.

Suppose our application has:

Frontend
Backend
Database

Instead of creating every Swarm service separately, we can define the complete application in a YAML file.

For example:

stack.yaml

Inside this file, we can define services, replicas, networks, volumes, ports, and other configurations.

Then we deploy:

```
docker stack deploy -c stack.yaml myapp
```

Docker Swarm reads the file and creates the required services.

We can check:

```
docker stack ls
```

We can check services:

```
docker stack services myapp
```

We can remove the stack:

```
docker stack rm myapp
```

Simple flow:

Stack YAML → docker stack deploy → Swarm Services → Application Running

Main point:

Docker Stack = Deploy a complete multi-service application on Docker Swarm using a YAML configuration file.

;;

20. What are Docker Secrets?

Docker Secrets are used to securely provide sensitive information to Docker Swarm services.

Suppose our database requires a password.

We should not directly place that password inside the Dockerfile or application image.

Instead, we can create a secret:

```
printf "mypassword" | docker secret create db_password -
```

Now the secret exists in the Swarm.

We can provide it to a service:

```
docker service create --name database --secret db_password mysql
```

The service can access the secret through Docker's secret mechanism.

Secrets are useful for:

Database passwords
API credentials
Private keys
Certificates
Other sensitive information

The main idea is to keep sensitive data separate from the application image and normal configuration.

Main point:

Docker Secrets = Secure way to provide sensitive information to Swarm services.

;;

21. What is Docker Healthcheck?

Docker Healthcheck is used to check whether the application inside a container is functioning correctly.

Suppose our container is running.
The container being in the running state does not always mean the application inside it is healthy.
For example, the application process may still exist but the application may not be responding correctly.
We can define a healthcheck in a Dockerfile:

```
HEALTHCHECK CMD curl --fail http://localhost:8080/ || exit 1
```

Docker periodically runs the healthcheck command.
If the command succeeds, the container is considered healthy.
If the check fails repeatedly, Docker can mark the container as unhealthy.
We can see the health status using:

```
docker ps
```

or:

```
docker inspect
```

Healthchecks are useful for determining the actual health of the application rather than only checking whether the container process is running.
Main point:
Docker Healthcheck = Checks whether the application inside the container is actually healthy.
;;

22. What are Docker Container Restart Policies?

Docker Restart Policies tell Docker what to do when a container stops or exits.
Suppose we have a production application container.
If the application unexpectedly stops, we may want Docker to start it again automatically.
For example:

```
docker run -d --restart unless-stopped --name myapp myimage
```

Docker provides restart policies such as:
no → Do not automatically restart.
on-failure → Restart when the container exits with a non-zero status.
always → Restart the container according to Docker's restart behavior.
unless-stopped → Restart the container unless it has been explicitly stopped.
For example:

```
docker run -d --restart always myapp
```

If the container stops unexpectedly, Docker attempts to restart it according to the selected policy.
This is useful for applications that should automatically recover from unexpected container exits.
Main point:
Restart Policy = Defines whether and when Docker should automatically restart a container.
;;

23. What are Docker Logs?

Docker Logs allow us to view the standard output and error output generated by a container's main process.
Suppose our backend application is running inside a container and an error occurs.
We can view the logs using:

```
docker logs mycontainer
```

Docker displays the captured output from the container's main process.
If we want to continuously watch new logs:

```
docker logs -f mycontainer
```

If we want only the latest logs:

```
docker logs --tail 100 mycontainer
```

In Docker Compose, we can use:

```
docker compose logs
```

or:

```
docker compose logs backend
```

Logs are very useful for troubleshooting application errors, startup problems, and runtime information.
Main point:
Docker Logs = View output generated by a container's main process.
;;

24. What is Docker Exec?

docker exec is used to execute a command inside an already running container.
Suppose our container is running:

```
docker ps
```

Now we want to open a shell inside the container.
We can use:

```
docker exec -it mycontainer sh
```

Now we get an interactive shell inside the running container.
We can also execute a specific command:

```
docker exec mycontainer ls /app
```

This runs ls /app inside the container.
Another example:

```
docker exec mycontainer env
```

This displays environment variables inside the container.
Docker Exec is mainly useful for debugging, checking files, inspecting configuration, and troubleshooting a running container.
Main point:
Docker Exec = Execute a command inside an already running container.
;;

25. What is Docker Architecture?

Docker Architecture explains how the different Docker components work together to build, run, and manage containers.

The main components are:

Docker Client → Used by the user to send Docker commands.

Docker Daemon → Performs Docker operations and manages Docker objects.

Docker Registry → Stores and distributes Docker images.

EXAMPLE:

Suppose we run:

```
docker run nginx
```

The Docker Client sends this request to the Docker Daemon.

The Docker Daemon checks whether the nginx image is available locally.

If the image is not available, Docker can pull it from a registry.

After getting the image, the Docker Daemon creates and starts the container.

Simple flow:

Docker Client → Docker Daemon → Docker Image → Docker Container

If image is not available locally:

Docker Client → Docker Daemon → Docker Registry → Pull Image → Create Container

Main point:

Docker Client = Sends commands.

Docker Daemon = Performs Docker operations.

Docker Registry = Stores and distributes images.

;;;

26. What is Docker Engine?

Docker Engine is the core technology used to build and run containers and manage Docker objects.

It provides the functionality required for operations such as:

Building images

Creating containers

Starting containers

Stopping containers

Managing networks

Managing volumes

EXAMPLE:

Suppose we run:

```
docker run nginx
```

Docker's client communicates with the Docker daemon, and the engine performs the required container operation.

The Docker Engine is therefore the core part that makes Docker container operations possible.

Main point:

Docker Engine = Core Docker technology used to build, run, and manage containers and related Docker objects.

;;;

27. What is Docker Client?

Docker Client is the command-line interface through which users communicate with Docker.

When we run:

```
docker build
docker run
docker ps
docker stop
docker pull
```

we are using the Docker Client.

The Docker Client sends these commands to the Docker Daemon.

EXAMPLE:

Suppose we run:

```
docker run nginx
```

The Docker Client sends the request.

The Docker Daemon receives the request and performs the actual operation.

So the user mainly interacts with Docker through the Docker Client.

Main point:

Docker Client = Interface used by the user to send Docker commands.

;;;

28. What is Docker Daemon?

Docker Daemon is the background service responsible for managing Docker objects and performing Docker operations.

It handles tasks such as:

Building images

Creating containers

Starting containers

Stopping containers

Managing networks

Managing volumes

EXAMPLE:

Suppose we run:

```
docker run nginx
```

The Docker Client sends the request to the Docker Daemon.

The daemon checks for the image, pulls it if necessary, creates the container, configures it, and starts it.

So the client is mainly used to send commands, while the daemon performs the Docker management work.

Main point:

Docker Daemon = Background service that performs and manages Docker operations.

;;

29. What is Docker Images vs Containers?

Docker Image and Docker Container are related, but they are not the same thing.

A Docker Image is a packaged, read-only template containing the application and its required dependencies.

A Docker Container is a running or stopped instance created from that image.

EXAMPLE:

Suppose we have:

myapp → Docker Image

We create:

```
docker run -d --name app1 myapp
```

Now:

app1 → Container

We can create another:

```
docker run -d --name app2 myapp
```

Now:

app2 → Container

Both containers are created from the same image.

So:

One Image → Multiple Containers

The image acts as the template, while the container is the actual instance where the application runs.

Simple flow:

Dockerfile → Image → Container

Main point:

Image = Template.

Container = Instance created from the template.

;;

30. What is Docker Compose vs Docker Swarm?

Docker Compose and Docker Swarm can both work with multiple services, but they are designed for different purposes.

Docker Compose is used to define and manage multiple containers/services for an application, commonly on a single Docker host or for development and testing environments.

Docker Swarm is a container orchestration technology used to manage services across multiple Docker hosts in a cluster.

EXAMPLE:

Suppose our shopping application has:

Frontend

Backend

Database

With Docker Compose, we can define these services inside:

compose.yaml

Then run:

```
docker compose up -d
```

Docker Compose creates and manages the required containers together.

Now suppose we have multiple production servers:

Server 1

Server 2

Server 3

We can create a Docker Swarm cluster and deploy services across those nodes.

For example:

```
docker service create --name web --replicas 3 nginx
```

Swarm manages those replicas across the cluster.

Simple comparison:

Docker Compose → Manage multiple application containers/services together.

Docker Swarm → Orchestrate services across multiple Docker hosts.

Main point:

Compose = Multi-container application management.

Swarm = Cluster-level container orchestration.

---

31. What is Docker vs Virtual Machine?

Docker and Virtual Machines are both used to isolate and run applications, but they work differently.

A Virtual Machine virtualizes a complete operating system. Each VM normally contains its own guest operating system, libraries, applications, and virtual hardware environment.

Docker containers share the host operating system kernel while isolating the application processes and their user-space dependencies.

EXAMPLE:

Suppose we have one physical server.

With Virtual Machines, we can have:

Physical Server → Hypervisor → VM 1 → Guest OS → Application

Physical Server → Hypervisor → VM 2 → Guest OS → Application

Each VM has its own guest operating system.

With Docker:

Physical Server → Docker Engine → Container 1 → Application

Physical Server → Docker Engine → Container 2 → Application

The containers share the host kernel instead of each carrying a complete guest operating system.

Because containers do not need a separate full guest operating system for each application, they are generally lighter and can start faster than traditional VMs, although the exact resource usage depends on the workload and configuration.

Virtual Machines provide stronger isolation at the hardware/OS virtualization level, while containers provide process-level isolation and are commonly used for packaging and deploying applications.

Simple comparison:

Virtual Machine → Full Guest OS → Application

Docker Container → Shared Host Kernel → Application

Main point:

VM = Virtualizes a complete machine/OS environment.

Docker Container = Isolates application processes while sharing the host kernel.

---

32. What is Docker Container Lifecycle?

Docker Container Lifecycle explains the different states and operations a container goes through from creation until removal.

Suppose we have a Docker Image called:

myapp

First, we can create a container:

```
docker create --name mycontainer myapp
```

Now the container is created but not running.

We can start it:

```
docker start mycontainer
```

Now the application is running.

We can check it:

```
docker ps
```

If we want to stop it:

```
docker stop mycontainer
```

Now the container is stopped, but the container still exists.

We can start the same container again:

```
docker start mycontainer
```

There is no need to create another container just to start the stopped one.

If we want to remove the container:

```
docker rm mycontainer
```

Now the container is removed.

If we want to create a completely new container from the image:

```
docker run -d --name mycontainer myapp
```

A new container is created and started.

Simple flow:

Docker Image → Create Container → Start → Running → Stop → Start Again → Remove

Important difference:

docker create → Creates container but does not start it.

docker start → Starts an existing stopped container.

docker stop → Stops a running container.

docker rm → Removes the container.

docker run → Creates and starts a new container.

Main point:

A Docker container can be created, started, stopped, started again, and finally removed. The Docker Image remains the template from which new containers can be created.

ChatGPT is AI and can make mistakes.

You’ll get smarter responses and can upload files, images, an